Exercices - Série 2, parties 1 et 2

420-249-AH - Programmation mobile iOS

1. Exercice 1 - Gestion d'un parc de véhicules

On souhaite créer une application qui gère plusieurs types de véhicules.

Créer d'abord l'énumération suivante :

```
enum TypeCarburant {
    case essence
    case diesel
    case electrique
    case hybride
}
```

Créer ensuite une classe de base :

```
class Vehicule
```

avec les propriétés suivantes :

```
marque: String
modele: String
prix: Double
typeCarburant: TypeCarburant
```

Ajouter une méthode :

```
func description() -> String
```

qui retourne les informations principales du véhicule.

Créer ensuite deux sous-classes :

```
class Voiture: Vehicule
class Camion: Vehicule
```

Sous-classe	Propriété ajoutée
`Voiture`	nombrePortes: Int
`Camion`	capaciteCharge: Double

Redéfinissez description() dans les deux sous-classes afin d'ajouter les informations spécifiques.

Ensuite :

1. Créer les véhicules

Créer au moins 3 voitures et 2 camions.

2. Regrouper dans un tableau

Les placer dans un seul tableau :

```
var vehicules: [Vehicule]
```

3. Observer le polymorphisme

Parcourir le tableau et appeler description() sur chaque objet afin d'observer le polymorphisme.

4. Protocole Vendable

Créer un protocole :

```
protocol Vendable
```

contenant :

```
var prix: Double { get }
func prixAvecTaxes() -> Double
```

Faire en sorte que Vehicule respecte ce protocole.

5. Extension du protocole

Ajouter une extension au protocole Vendable qui fournit une méthode :

```
func prixFormate() -> String
```

La méthode doit retourner une chaîne comme :

```
Console
Prix : 35 000.00 $
```

6. Filtrer par prix

Utiliser filter pour obtenir uniquement les véhicules dont le prix est inférieur à 40 000 $.

7. Filtrer par carburant

Utiliser filter pour obtenir uniquement les véhicules électriques.

8. Trier par prix

Utiliser sorted pour classer tous les véhicules du moins cher au plus cher.

9. Créer une liste de descriptions courtes

Utiliser map pour créer un nouveau tableau contenant uniquement les marques et modèles sous cette forme :

```
Console
Toyota Corolla
Ford F-150
```

10. Calculer la valeur totale

Utiliser reduce pour calculer la valeur totale de tous les véhicules du parc.

11. Créer une closure de rabais

Créer une closure :

```
let appliquerRabais: (Double, Double) -> Double
```

Elle reçoit un prix et un pourcentage de rabais, et retourne le nouveau prix.

12. Appliquer le rabais

Utiliser cette closure pour calculer le prix d'un véhicule après un rabais de 15 %.

2. Exercice 2 - Gestion de commandes de livraison

On souhaite développer une application qui gère différentes méthodes de livraison.

Créer l'énumération suivante :

```
enum StatutLivraison {
    case enPreparation
    case expediee
    case livree
    case annulee
}
```

Créer ensuite un protocole :

```
protocol Livrable
```

qui impose :

```
var destination: String { get }
var cout: Double { get }
func livrer()
```

Créer une classe de base :

```
class Livraison
```

avec les propriétés :

```
destination: String
cout: Double
statut: StatutLivraison
```

Faire en sorte que cette classe respecte le protocole Livrable.

Créer ensuite deux sous-classes :

```
class LivraisonStandard: Livraison
class LivraisonExpress: Livraison
```

Sous-classe	Propriété ajoutée
LivraisonStandard	delaiJours: Int
LivraisonExpress	heuresEstimees: Int

Redéfinir la méthode :

```
func livrer()
```

dans chaque sous-classe afin d'afficher un comportement différent.

Exemple :

```
Console
Livraison standard vers Montréal dans 3 jours.
Livraison express vers Laval dans 6 heures.
```

Puis :

1. Créer les livraisons

Créer au moins 5 livraisons différentes, en mélangeant les deux types.

2. Regrouper dans un tableau

Les placer dans :

```
var livraisons: [Livraison]
```

3. Observer le polymorphisme

Parcourir le tableau et appeler livrer() sur chaque objet afin de démontrer le polymorphisme.

4. Extension de l'énumération

Ajouter une extension à StatutLivraison :

```
var description: String
```

Elle doit retourner :

Cas	Description
`.enPreparation`	En préparation
`.expediee`	Expédiée
`.livree`	Livrée
`.annulee`	Annulée

5. Extension de la classe

Ajouter une extension à Livraison avec une méthode :

```
func resume() -> String
```

Elle doit retourner une phrase contenant la destination, le coût et le statut.

6. Filtrer par statut

Utiliser filter pour obtenir seulement les livraisons dont le statut est .livree.

7. Filtrer par coût

Utiliser filter pour obtenir uniquement les livraisons dont le coût est supérieur à 20 $.

8. Trier par coût

Utiliser sorted pour classer les livraisons du coût le plus élevé au moins élevé.

9. Extraire les destinations

Utiliser map pour créer un tableau contenant seulement les destinations.

Exemple :

```
["Montréal", "Laval", "Québec"]
```

10. Calculer le coût total

Utiliser reduce pour calculer le coût total de toutes les livraisons.

11. Créer une closure de frais

Créer une closure :

```
let fraisSupplementaires: (Double) -> Double
```

Cette closure ajoute 5 % au coût reçu.

12. Appliquer les frais avec map

Utiliser cette closure avec map afin de créer un nouveau tableau contenant les coûts après ajout des frais supplémentaires.

13. Trier selon la longueur du nom

Créer une autre closure permettant de trier les livraisons selon la longueur du nom de destination.

Utiliser cette closure avec :

```
sorted(by:)
```

14. Afficher le résumé final

Afficher finalement, pour chaque livraison, un résumé similaire à :

```
Console
Montréal | Express | 25.00 $ | Livrée
```